Week 6 · Lab 2

Version C

Graded Practice:

Loops and First Functions

10 problems · 10 points each · checked by the TA at your desk

Week: 6 — second session (graded)

Duration: 2 hours 15 minutes

Format: individual problem solving in CLion

Covers: Lecture 5 — `do-while`, `for`, nested loops, and writing your own functions (return type, parameters, `return`)

Name:

Group:

Date:

How this lab works

1. Solve the problems in CLion, one `.cpp` file per problem. Every program reads its input with `cin` and prints its answer with `cout`. **New this week:** write the solution as the *function* named in the problem, above `main`; `main` reads the input, calls the function and prints the result.
2. **When a problem works, raise your hand.** The TA comes to your desk, reads your code, runs a few inputs and writes your points in the box under the problem. Nothing is uploaded.
3. There are **10 problems, 10 points each** (100 in total): problems 1–7 are **easy** and 8–10 **hard**. Solve them in any order; a problem with no attempt earns 0 points. There is enough work for the whole session — keep going until the end.
4. Under each title, **Practises:** says which part of Lecture 5 the problem trains.
5. You may ask the TA for help — this is practice, not an exam. The TA may ask you a short question about your code; be ready to explain *why* it works.

Quick reference

How every solution looks this week the function above `main`; `main` reads, calls and prints.

```cpp
#include <iostream>
using namespace std;

int countDigits(int n) {        // the function
    ...
    return count;
}

int main() {
    int n;
    cin >> n;
    cout << countDigits(n) << endl;
    return 0;
}
```

A `bool` function is printed with `cout << (f(n) ? "true" : "false");`. In the answer key the `#include` lines are left out.

Functions a `void` function only does something (prints) and is called on its own line, `f(n);`. `return` hands a value back *and ends the function* — a `return` inside a loop stops the loop too. `return` is not `cout`: compute in the function, print in `main`. A parameter is a copy; changing it does not change the caller's variable. A function must be written above the place where it is called.

`for`, `while`, `do-while`

```cpp
for (int i = 1; i <= n; i++) { ... }   // n passes
for (long long k = 1; k * k <= n; k++)  // k*k may pass 2^31
while (n > 0) { d = n % 10; n /= 10; } // digits
do { ... } while (cond);               // at least once
```

Nested loops multiply: outer × inner passes. A loop inside a function that is called in a loop is a nested loop too. For patterns: the inner loop prints one row, `endl` goes *after* it.

Big numbers `int` holds about $\pm 2.1 \cdot 10^9$, `long long` about $\pm 9.2 \cdot 10^{18}$. Factorials, sums of squares and products grow fast: make the accumulator *and* the values you multiply `long long`.

`<cmath>` `sqrt`, `pow`, `abs`, `floor`, `ceil`, `round` return `double`. Some problems forbid them — then write the loop yourself.

Classic bugs `for (...);` with a stray `;`; `<` versus `<=`; `i * i` overflowing in `int`; `int a = b; b = a + b;` without a temporary; printing inside the function *and* in `main`.

EASY

Problems 1–7 · 70 points

Short programs: one or two ideas each. Test every branch, not just the examples.

1 Ugly Number 10 pts

Practises: `while` loops that divide, inside `bool isUgly(int n)`.

An *ugly number* is a positive integer whose only prime factors are 2, 3 and 5 (1 counts as ugly). Read an `int` n (negative values included) and print `true` if it is ugly, otherwise `false`.

Input	Output	Why
`6`	`true`	$2 \cdot 3$.
`1`	`true`	No prime factors at all.
`14`	`false`	$2 \cdot 7$.

Hint: divide out every 2, every 3 and every 5. What must be left?

TA: ____ / 10 pts initials ____

2 Product of Digits 10 pts

Practises: a `while` loop over the digits, inside `int digitProduct(int n)`.

Read n ($0 \le n \le 10^9$) and print the product of its digits. Write `int digitProduct(int n)` and call it from `main`. Careful: the number 0 has one digit, 0.

Input	Output	Why
`345`	`60`	$3 \cdot 4 \cdot 5$.
`2026`	`0`	A digit is 0.
`7`	`7`	

TA: ____ / 10 pts initials ____

3 Sum of a Range 10 pts

Practises: a `for` loop with `long long` inside `long long sumRange(int a, int b);` parameters as copies.

Read two integers a and b ($-10^6 \le a, b \le 10^6$, in any order) and print the sum of all integers between them, both included. Write `long long sumRange(int a, int b);` if $a > b$, swap them inside the function first.

Input	Output	Why
`3 7`	`25`	$3+4+5+6+7$.
`7 3`	`25`	Same numbers.
`-2 2`	`0`	

TA: ____ / 10 pts initials ____

4 Smallest Integer Divisible by K 10 pts

Practises: a `for` loop that keeps only a remainder.

Read k ($1 \le k \le 10^5$). Find the length of the smallest number made only of 1s ($1, 11, 111, \ldots$) that is divisible by k, or print -1 if there is none. Write `int smallestRepunitDivByK(int k)`.

Input	Output	Why
1	1	
2	-1	Numbers of 1s are odd.
3	3	$111 = 3 \cdot 37$.

Hint: these numbers overflow quickly, so keep only their remainder modulo k: the next one is $(r \cdot 10 + 1) \% k$. There are only k possible remainders, so after k steps without 0 it never comes.

TA: ____ / 10 pts initials ____

5 Twin Primes 10 pts

Practises: a `bool` function called twice in one condition, inside a `for` loop.

Twin primes are two primes that differ by 2, like 11 and 13. Write `bool isPrime(int n)` (test d while $d \cdot d \le n$). `main` reads N ($1 \le N \le 10\,000$) and prints all twin-prime pairs $(p, p+2)$ with $p + 2 \le N$ on one line, and then how many there are.

Example. Input `20` prints:

```
(3,5) (5,7) (11,13) (17,19)
count: 4
```

TA: ____ / 10 pts initials ____

6 Largest and Smallest 10 pts

Practises: two tiny functions called inside a `for` loop; starting values.

Write `int max2(int a, int b)` and `int min2(int a, int b)`, which return the larger and the smaller of two numbers. `main` reads n ($1 \le n \le 1000$) and then n integers (any sign), and prints the largest and the smallest of them, keeping both up to date with `max2` and `min2` as it reads — without storing the numbers.

Input	Output	Why
`5 4 9 1 7 3`	`max=9 min=1`	
`3 -5 -2 -9`	`max=-2 min=-9`	All negative.
`1 42`	`max=42 min=42`	

Hint: Start `mx` *and* `mn` *from the* first *number read, not from 0.*

TA: ____ / 10 pts initials ____

7 Chessboard 10 pts

Practises: two nested `for` loops and a function that returns a `char`.

Write `char cellAt(int row, int col)`, which returns `'#'` when `row + col` is even and `'.'` otherwise (rows and columns counted from 0). `main` reads the number of rows and columns (1 to 30 each) and prints the board.

Example. Input `3 5` prints:

```
#.#.#
.#.#.
#.#.#
```

TA: ____ / 10 pts initials _____

HARD

Problems 8–10 · 30 points

Longer programs: several loops and functions working together. Read the hint before you start.

8 Approximating e 10 pts

Practises: a function returning `double`, called in a `for` loop; output with `setprecision`.

The number $e = 2.71828\ldots$ is the sum $\frac{1}{0!} + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \cdots$ Write `double factorial(int k)`. `main` reads n ($1 \le n \le 20$), adds the first n terms and prints the sum with 10 digits after the point (`fixed`, `setprecision(10)` from `<iomanip>`).

Input	Output	Why
`1`	`1.0000000000`	Only $1/0!$.
`3`	`2.5000000000`	$1 + 1 + 0.5$.
`10`	`2.7182815256`	

TA: ____ / 10 pts initials _____

9 Strong Numbers 10 pts

Practises: one function calling another, both with loops, called from a `for` loop in `main`.

A number is *strong* if it equals the sum of the factorials of its digits: $145 = 1! + 4! + 5! = 1 + 24 + 120$. Write `int factorial(int d)` and `int digitFactorialSum(int n)`, which uses `factorial` for every digit. `main` reads N ($1 \le N \le 100\,000$) and prints all strong numbers from 1 to N on one line.

Input	Output	Why
`200`	`1 2 145`	$1 = 1!$, $2 = 2!$.
`1`	`1`	

TA: ____ / 10 pts initials _____

10 Floyd's Triangle 10 pts

Practises: a `void` function with two parameters, called from a loop that keeps a running value.

Floyd's triangle writes $1, 2, 3, \ldots$ in rows: row r holds r numbers. Write `void printRow(int start, int count)`, which prints `count` consecutive numbers beginning at `start`, separated by spaces, and ends the line. `main` reads n ($1 \le n \le 20$) and prints the first n rows.

Example. Input 4 prints:

```
1
2 3
4 5 6
7 8 9 10
```

TA: ____ / 10 pts initials _____

Score sheet — Version C

	Problem	Level	Points	Earned	TA
1	Ugly Number	Easy	10		
2	Product of Digits	Easy	10		
3	Sum of a Range	Easy	10		
4	Smallest Integer Divisible by K	Easy	10		
5	Twin Primes	Easy	10		
6	Largest and Smallest	Easy	10		
7	Chessboard	Easy	10		
8	Approximating e	Hard	10		
9	Strong Numbers	Hard	10		
10	Floyd's Triangle	Hard	10		
	Total		**100**		

After the lab

Next week: prototypes, references and overloading.

End of Week 6 · Lab 2 · Version C